

3.3. Widgets y componentes en Flutter



DESARROLLO DE APLICACIONES WEB MULTIPLATAFORMA NIVEL 3.

UNIDAD 3:

INTRODUCCION A FLUTTER

Contenido

Introducción.....	3
Importancia de los widgets y componentes en la construcción de interfaces de usuario	3
Diferencias entre widgets y componentes.....	4
Widgets.....	5
Creación y uso de widgets en Flutter	5
Ciclo de vida y actualización de widgets	9
Optimización del rendimiento de los widgets	9
Tipos de widgets en Flutter	10
Componentes	13
Creación y uso de componentes en Flutter	13
Ejemplo práctico de creación de un componente en Flutter	14
Ejemplos de componentes en Flutter:	18
Ciclo de vida de los componentes	18
Etapas del ciclo de vida:	18
Métodos del ciclo de vida.....	19
Ejemplo de ciclo de vida:	19
Tipos de componentes:.....	19
Recursos adicionales.....	20

Introducción

Flutter es un framework de desarrollo de aplicaciones móviles que utiliza un enfoque basado en widgets para construir interfaces de usuario (UI).

Como hemos visto en la unidad didáctica anterior, los widgets son los bloques de construcción fundamentales de las interfaces de Flutter, y se pueden utilizar para crear cualquier elemento visual, desde botones y texto hasta menús y animaciones.

Un componente, por otro lado, es un widget o una agrupación de widgets que se combina para crear una unidad funcional reutilizable. Los componentes permiten modularizar el código y crear interfaces de usuario más complejas y organizadas.

En resumen, los widgets son los elementos básicos de la UI en Flutter, mientras que los componentes son unidades funcionales reutilizables compuestas por widgets.

Importancia de los widgets y componentes en la construcción de interfaces de usuario

Los widgets y componentes son esenciales para la construcción de interfaces de usuario en Flutter por las siguientes razones:

- **Modularidad:** Permiten dividir las interfaces de usuario en partes más pequeñas y manejables, lo que facilita el desarrollo y mantenimiento del código.
- **Reutilización:** Los componentes se pueden reutilizar en diferentes partes de la aplicación, lo que ahorra tiempo y esfuerzo al desarrollador.
- **Flexibilidad:** Los widgets y componentes son muy flexibles y se pueden personalizar para crear una amplia variedad de interfaces de usuario.
- **Rendimiento:** Flutter optimiza el rendimiento de los widgets, lo que garantiza que las interfaces de usuario sean fluidas y receptivas.

Diferencias entre widgets y componentes

La principal diferencia entre widgets y componentes es que los widgets son elementos básicos de la UI, mientras que los componentes son unidades funcionales reutilizables compuestas por widgets.

Otras diferencias clave incluyen:

- Nivel de abstracción: Los widgets son más abstractos que los componentes, ya que representan elementos visuales individuales. Los componentes, por otro lado, encapsulan una lógica y funcionalidad específicas.
- Ciclo de vida: Los widgets tienen un ciclo de vida propio que se gestiona por Flutter. Los componentes, al ser agrupaciones de widgets, heredan el ciclo de vida de los widgets que los componen.
- Reutilización: Los widgets se pueden reutilizar individualmente, mientras que los componentes se reutilizan como unidades completas.

En general, los widgets y componentes son herramientas esenciales para la construcción de interfaces de usuario en Flutter. Al comprender sus diferencias y utilizarlos de manera efectiva, los desarrolladores pueden crear interfaces de usuario hermosas, funcionales y fáciles de mantener.

Widgets



Abordemos ahora con más detalle aspectos de los widgets:

Creación y uso de widgets en Flutter

Como ya sabes los widgets son bloques de construcción reutilizables que permiten crear desde elementos simples como texto e imágenes hasta interfaces complejas y dinámicas.

En este epígrafe, exploraremos a fondo la creación y el uso de widgets en Flutter, abarcando desde la declaración básica hasta la gestión de ciclos de vida y la optimización del rendimiento.

Construcción de interfaces con widgets

Los widgets en Flutter se declaran utilizando el lenguaje de programación Dart. Cada widget tiene una clase asociada que define sus propiedades, métodos y comportamiento.

Esta clase proporciona un modelo para crear instancias de widgets con características específicas.

- Clase
Una clase de widget en Flutter tiene típicamente la siguiente estructura:

Dart

```
class MiWidget extends StatelessWidget {
```

```
// Propiedades

final String titulo;

final Color color;


// Constructor

const MiWidget({
  Key? key,
  required this.titulo,
  required this.color,
}) : super(key: key);


// Método build

@override
Widget build(BuildContext context) {
  return Container(
    padding: const EdgeInsets.all(16.0),
    decoration: BoxDecoration(
      color: color,
      borderRadius: BorderRadius.circular(10.0),
    ),
    child: Text(
      titulo,
      style: TextStyle(
        fontSize: 20.0,
        fontWeight: FontWeight.bold,
```

```
        color: Colors.white,
    ),
),
);
}
}
```

Este código define un widget reutilizable que muestra un título dentro de un contenedor de color con esquinas redondeadas y relleno. El color de fondo y el texto del título se pueden personalizar proporcionando valores a las propiedades color y titulo al crear una instancia de este widget.

- Instancia

Para crear una instancia de la clase MiWidget en base al código proporcionado anteriormente, deberá llamar al constructor y pasar los valores necesarios para las propiedades título y color.

Aquí hay un ejemplo de cómo hacerlo:

```
Dart

MiWidget miWidget = MiWidget(
    titulo: "Mi título personalizado",
    color: Colors.blue,
);
```

En este ejemplo:

- miWidget es una variable de tipo MiWidget que contendrá la instancia del widget personalizado.
- MiWidget(...) es la llamada al constructor que crea una instancia de la clase MiWidget.
- titulo: "Mi título personalizado" pasa la cadena "Mi título personalizado" a la propiedad titulo del constructor. Esto establecerá el texto del título que se mostrará en el widget.

- color: Colors.blue pasa la constante Colors.blue a la propiedad color del constructor. Esto establecerá el color de fondo del widget en azul.

Propiedades y atributos de los widgets

Como has visto en el ejemplo anterior, los widgets tienen propiedades y atributos que configuran su apariencia y comportamiento. Estas propiedades y atributos se establecen al crear la instancia del widget, utilizando pares clave-valor.

Jerarquía de widgets y árbol de widgets

Recordando contenidos de anteriores unidades didácticas, los widgets se organizan en una jerarquía anidada, formando un árbol de widgets.

La raíz del árbol es la aplicación en sí, y cada widget puede tener uno o más hijos. La estructura del árbol de widgets define la disposición de los elementos en la interfaz.

```
Scaffold(
  appBar: AppBar(
    title: Text('Mi Aplicación'),
  ),
  body: Center(
    child: Column(
      children: <Widget>[
        Text('Widget 1'),
        Text('Widget 2'),
        Text('Widget 3'),
      ],
    ),
  ),
);
```


Usa el código con precaución.

Composición de widgets para crear interfaces complejas

La composición de widgets es el principio fundamental para crear interfaces complejas en Flutter. Al combinar widgets simples en componentes, se pueden construir elementos más elaborados y con mayor funcionalidad.

Sobre los componentes hablaremos más adelante en esta unidad didáctica

Ciclo de vida y actualización de widgets

Cada widget en Flutter tiene un ciclo de vida propio que define cuándo se crea, se inserta en el árbol de widgets, se actualiza y se elimina.

- `initState`: Se llama cuando se crea el widget por primera vez.
- `build`: Se llama cada vez que el widget necesita ser reconstruido o actualizado.
- `didChangeDependencies`: Se llama cuando las dependencias del widget cambian.
- `didUpdateWidget`: Se llama cuando el widget recibe nuevas propiedades.
- `dispose`: Se llama cuando el widget se elimina del árbol de widgets.

Dependencias entre widgets y actualización de la interfaz

Los widgets pueden depender de otros widgets para su estado o apariencia. Cuando un widget dependiente cambia, el widget que depende de él se actualiza.

Flutter utiliza un mecanismo eficiente para detectar cambios y propagar las actualizaciones solo a los widgets que las necesitan, lo que garantiza un rendimiento óptimo.

Optimización del rendimiento de los widgets

Existen varias estrategias para optimizar el rendimiento de los widgets en Flutter:

- Evitar la reconstrucción innecesaria: Solo redibujar los widgets que realmente han cambiado.
- Utilizar widgets sin estado: Los widgets sin estado son más eficientes que los widgets con estado.

- Compartir código y estilos: Reutilizar código y estilos para evitar la creación redundante de objetos.
- Aprovechar las herramientas de depuración: Flutter ofrece herramientas para identificar cuellos de botella de rendimiento y optimizar el código.

Tipos de widgets en Flutter

Flutter clasifica los widgets en tres categorías principales:

- Widgets sin estado (Stateless Widgets): No mantienen ningún dato interno y su apariencia depende únicamente de sus propiedades y del estado de los widgets padre. Son ideales para elementos visuales estáticos como texto, imágenes o iconos.
- Widgets con estado (Stateful Widgets): Mantienen un estado interno que puede cambiar con el tiempo. Son útiles para elementos dinámicos que requieren actualizar la interfaz en respuesta a eventos o cambios en el estado.
- Widgets básicos (Basic Widgets): Son widgets predefinidos por Flutter que proporcionan funcionalidades básicas para la construcción de interfaces de usuario. Se pueden clasificar en subcategorías más específicas:
 - Widgets compuestos (Compound Widgets): Combinan varios widgets básicos para crear elementos más complejos.
 - Widgets de diseño (Layout Widgets): Definen la posición y organización de los widgets en la pantalla.
 - Widgets interactivos (Interactive Widgets): Permiten la interacción del usuario con la interfaz, como botones, formularios o entradas de texto.

Análisis en profundidad:

Veamos cada uno de estos widgets con un poco más de detalle:

- Widgets sin estado (Stateless Widgets):
Características:
 - Inmutables: Su apariencia no cambia con el tiempo.
 - Simples y eficientes: Adecuados para elementos estáticos.
 - Sin gestión de estado: No requieren almacenar datos internos.

Ejemplos:

- Text: Muestra texto en la pantalla.
- Image: Muestra una imagen.
- Icon: Muestra un icono.
- Container: Define un contenedor para otros widgets.
- Row: Organiza widgets en una fila horizontal.
- Column: Organiza widgets en una columna vertical.
- Widgets con estado (Stateful Widgets):
Características:
 - Mutables: Su apariencia puede cambiar con el tiempo.
 - Complejos: Gestionan un estado interno que afecta su apariencia.

Útiles para elementos dinámicos: Actualizan la interfaz en respuesta a eventos

Ejemplos:

- StatefulWidget: Clase base para widgets con estado.
- Ticker: Controla animaciones y actualizaciones periódicas.
- Form: Crea formularios para la entrada de datos.
- AnimatedWidget: Anima la apariencia de un widget en base a su estado.
- Widgets básicos (Basic Widgets):
Como se ha comentado proporcionan la posibilidad de construir funcionalidades básicas.

Subcategorías:

- Widgets compuestos (Compound Widgets):
 - Row: Organiza widgets en una fila horizontal.
 - Column: Organiza widgets en una columna vertical.
 - Stack: Posiciona widgets uno encima del otro.
 - Wrap: Ajusta widgets dentro de un espacio disponible.
 - CustomMultiChildLayout: Crea diseños personalizados.
- Widgets de diseño (Layout Widgets):
 - Scaffold: Proporciona la estructura básica de una pantalla.
 - AppBar: Crea una barra de título para la aplicación.
 - TabBar: Implementa una barra de pestañas.
 - Drawer: Añade un menú lateral deslizable.

- BottomNavigationBar: Crea una barra de navegación inferior.
- Widgets interactivos (Interactive Widgets):
 - ElevatedButton: Botón con efecto de elevación.
 - OutlinedButton: Botón con borde.
 - TextButton: Botón simple con texto.
 - IconButton: Botón con icono.
 - TextField: Campo de entrada de texto.
 - GestureDetector: Detecta gestos y eventos táctiles.

La clasificación de widgets en Flutter proporciona una estructura organizada para comprender y utilizar estos elementos esenciales en el desarrollo de interfaces de usuario. Al diferenciar entre widgets sin estado, con estado y básicos, y sus subcategorías, los desarrolladores pueden seleccionar el widget adecuado para cada caso de uso y crear interfaces eficientes y dinámicas.

Componentes



Centrémonos en la agregación de widgets o componentes:

Creación y uso de componentes en Flutter

Los pasos para crear componentes en Flutter son los siguientes:

1. Definir los widgets base: Comience por crear widgets simples con funcionalidades específicas, como botones, textos con formato o contenedores con estilos predefinidos. Estos widgets base servirán como bloques de construcción para crear componentes más complejos.
2. Crear widgets compuestos: Combine widgets base en estructuras jerárquicas para crear componentes con mayor funcionalidad. Por ejemplo, un widget TarjetaProducto podría estar compuesto por widgets Image, Text y Container para mostrar la imagen, el nombre y el precio de un producto.
3. Exponer propiedades y métodos: Proporcione propiedades y métodos a los widgets compuestos para permitir su configuración y control desde el exterior. Por ejemplo, un widget Formulario podría tener propiedades para definir los campos de entrada y un método para enviar el formulario.
4. Organizar y empaquetar: Utilice paquetes para organizar y distribuir sus componentes compuestos, facilitando su reutilización en diferentes proyectos. Flutter ofrece herramientas como flutter pub para crear y publicar paquetes.

Ejemplo práctico de creación de un componente en Flutter

Objetivo: Crear un componente reutilizable llamado TarjetaNoticia que muestre la imagen, el título y una breve descripción de una noticia.

Pasos:

1. Definir los widgets base:

Los widgets que combinaremos son:

- a. Widget imagenNoticia(String urlImagen): Este widget mostrará la imagen de la noticia. Recibirá la URL de la imagen como parámetro.

```
Widget imagenNoticia(String urlImagen) {  
    return Image.network(  
        urlImagen,  
        fit: BoxFit.cover,  
        height: 150,  
    );  
}
```

- b. Widget tituloNoticia(String titulo): Este widget mostrará el título de la noticia. Recibirá el título como parámetro.

Dart

```
Widget tituloNoticia(String titulo) {  
    return Text(  
        titulo,  
        style: TextStyle(  
            fontSize: 18,  
            fontWeight: FontWeight.bold,  
        ),  
    );  
}
```

```
}
```

- c. Widget `descripcionNoticia(String descripcion)`: Este widget mostrará una breve descripción de la noticia. Recibirá la descripción como parámetro.

Dart

```
Widget descripcionNoticia(String descripcion) {  
  return Text(  
    descripcion,  
    maxLines: 2,  
    overflow: TextOverflow.ellipsis,  
  );  
}
```

2. Crear el widget compuesto `TarjetaNoticia`:

El widget `TarjetaNoticia` combinará los widgets `base imagenNoticia`, `tituloNoticia` y `descripcionNoticia` para mostrar la información completa de una noticia.

Dart

```
class TarjetaNoticia extends StatelessWidget {  
  final String urlImagen;  
  final String titulo;  
  final String descripcion;  
  
  const TarjetaNoticia({  
    Key? key,  
    required this.urlImagen,  
    required this.titulo,  
    required this.descripcion,  
  }) : super(key: key);  
}
```

```
} : super(key: key);
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Container(
```

```
    padding: const EdgeInsets.all(16.0),
```

```
    margin: const EdgeInsets.only(bottom: 16.0),
```

```
    decoration: BoxDecoration(
```

```
      color: Colors.white,
```

```
      borderRadius: BorderRadius.circular(10.0),
```

```
      boxShadow: [
```

```
        BoxShadow(
```

```
          color: Colors.grey.withOpacity(0.2),
```

```
          spreadRadius: 3.0,
```

```
          blurRadius: 10.0,
```

```
          offset: Offset(0, 3.0),
```

```
        ),
```

```
      ],
```

```
    ),
```

```
    child: Column(
```

```
      crossAxisAlignment: CrossAxisAlignment.start,
```

```
      children: [
```

```
        imagenNoticia(urlImagen),
```

```
        SizedBox(height: 16.0),
```

```
        tituloNoticia(titulo),
```



```
        SizedBox(height: 8.0),
        descripcionNoticia(descripcion),
      ],
    ),
  );
}
}
```

3. Exponer propiedades y métodos:

El widget TarjetaNoticia expone las propiedades urlImagen, titulo y descripcion para recibir la información de la noticia.

En la creación de componentes en Flutter, la exposición de propiedades y métodos es crucial para permitir su configuración y control desde el exterior. Esto significa que los widgets compuestos pueden recibir datos, modificar su estado o realizar acciones en respuesta a eventos externos.

4. Organizar y empaquetar:

Para organizar y distribuir este componente, se puede crear un paquete Flutter y publicarlo en repositorios como Pub.dev. Esto permitirá reutilizarlo en otros proyectos.

Uso del componente TarjetaNoticia:

Dart

```
TarjetaNoticia(
  urlImagen: 'https://ejemplo.com/imagen.jpg',
  titulo: 'Título de la noticia',
  descripcion: 'Breve descripción de la noticia',
)
```

Usa el código con precaución.

Ejemplos de componentes en Flutter:

- Barra de navegación: Una barra de navegación superior se puede componer con widgets `IconButton` para las opciones de menú y un widget `Text` para el título de la aplicación.
- Lista de elementos: Una lista de elementos se puede componer con widgets `ListTile` para cada elemento, cada uno con widgets `Image`, `Text` y `Icon` para mostrar la información del elemento.
- Formulario de registro: Un formulario de registro se puede componer con widgets `TextField` para los campos de entrada, un widget `DropDownButton` para seleccionar opciones y un widget `ElevatedButton` para enviar el formulario.

Ciclo de vida de los componentes

El ciclo de vida de los componentes en Flutter define las etapas por las que pasa un widget desde su creación hasta su destrucción. Comprender este ciclo es crucial para crear interfaces de usuario dinámicas y eficientes.

Etapas del ciclo de vida:

- Creación: El widget se crea por primera vez y se le asigna un estado. El método `initState()` se llama para inicializar el estado del widget.
- Montaje: El widget se inserta en el árbol de widgets y se muestra en la interfaz de usuario. El método `build()` se llama para generar el widget y sus elementos.
- Actualización: El widget se actualiza debido a cambios en el estado o en la entrada del usuario. El método `build()` se vuelve a llamar para reflejar los cambios.
- Desactivación: El widget se retira temporalmente de la interfaz de usuario, pero aún mantiene su estado. El método `didChangeDependencies()` se llama para notificar al widget sobre los cambios en sus dependencias.
- Reactivación: El widget se vuelve a mostrar en la interfaz de usuario. El método `didChangeDependencies()` se llama nuevamente para notificar al widget sobre los cambios en sus dependencias.
- Destrucción: El widget se elimina permanentemente del árbol de widgets y su estado se libera. El método `dispose()` se llama para limpiar cualquier recurso utilizado por el widget.

Métodos del ciclo de vida:

- `initState()`: Se llama una vez cuando se crea el widget y se le asigna un estado.
- `build()`: Se llama cada vez que el widget necesita generar su representación en la interfaz de usuario.
- `didChangeDependencies()`: Se llama cuando las dependencias del widget cambian, como el estado de sus padres o la configuración de la aplicación.
- `setState()`: Se utiliza desde el método `build()` para marcar el widget como sucio, lo que desencadena una nueva llamada a `build()` para actualizar la interfaz de usuario.
- `dispose()`: Se llama cuando el widget se destruye y se libera su estado.

Ejemplo de ciclo de vida:

Considera un widget Contador que muestra un número y tiene un botón para incrementarlo.

- `initState()`: Inicializa el valor del contador a 0.
- `build()`: Muestra el número actual y el botón.
- `increment()`: Incrementa el valor del contador y llama a `setState()` para actualizar la interfaz de usuario.
- `dispose()`: Cancela cualquier temporizador o suscripción asociados con el widget.

Tipos de componentes:

Los más destacados son los siguientes:

- `AppBar`: Barra de título de la aplicación
- `Scaffold`: Estructura básica de una pantalla en Flutter
- `BottomNavigationBar`: Barra de navegación inferior
- `Drawer`: Menú lateral deslizable
- `ListView`: Lista de elementos desplazable
- `GridView`: Cuadrícula de elementos
- `Text`: Visualización de texto en la interfaz
- `Image`: Visualización de imágenes en la interfaz
- `Icon`: Visualización de iconos en la interfaz
- `Button`: Botones para la interacción del usuario
- `Form`: Creación de formularios para la entrada de datos

Recursos adicionales

- Documentación oficial de Flutter: <https://docs.flutter.dev/>
- Tutoriales de Flutter: <https://developers.google.com/learn/pathways/intro-to-flutter>
- Ejemplos de código de Flutter: <https://github.com/flutter/samples>
- Comunidad de Flutter en español: <https://esflutter.dev/>

¡Enhorabuena por completar esta unidad didáctica sobre widgets y componentes en Flutter!

En este recorrido, hemos explorado los fundamentos de la creación de interfaces de usuario en Flutter, aprendiendo sobre:

- La importancia de los widgets: Los bloques de construcción básicos de la interfaz de usuario en Flutter.
- Los diferentes tipos de widgets: Desde widgets simples como Text y Image hasta widgets compuestos más complejos.
- La composición de widgets: La técnica fundamental para crear interfaces de usuario jerárquicas y reutilizables.
- Los componentes: Unidades reutilizables de código que encapsulan una funcionalidad específica o una parte de la interfaz de usuario.
- El ciclo de vida de los componentes: Las etapas por las que pasa un widget desde su creación hasta su destrucción.

Esperamos que esta unidad didáctica te haya brindado una base sólida para comprender el desarrollo de interfaces de usuario en Flutter.

Recuerda que la práctica es esencial para dominar este tema. Continúa creando widgets y componentes, experimentando con diferentes diseños y explorando las diversas herramientas y recursos disponibles para Flutter.

¡Atrévete a crear interfaces de usuario dinámicas, atractivas y eficientes con Flutter!

¡Hasta la próxima!